

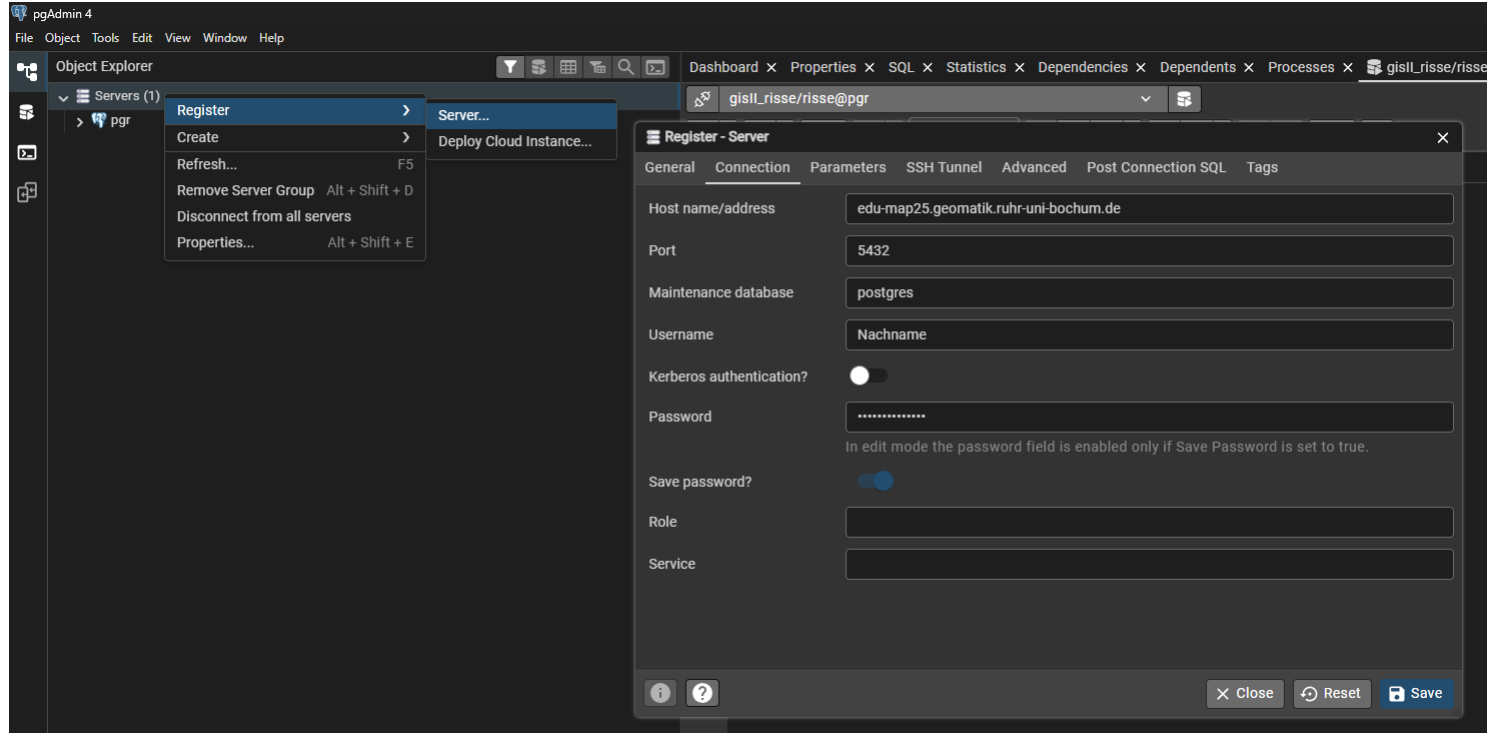
TITELBILD

RUHR-UNIVERSITÄT BOCHUM

ÜBUNG pgROUTING TUTORIAL

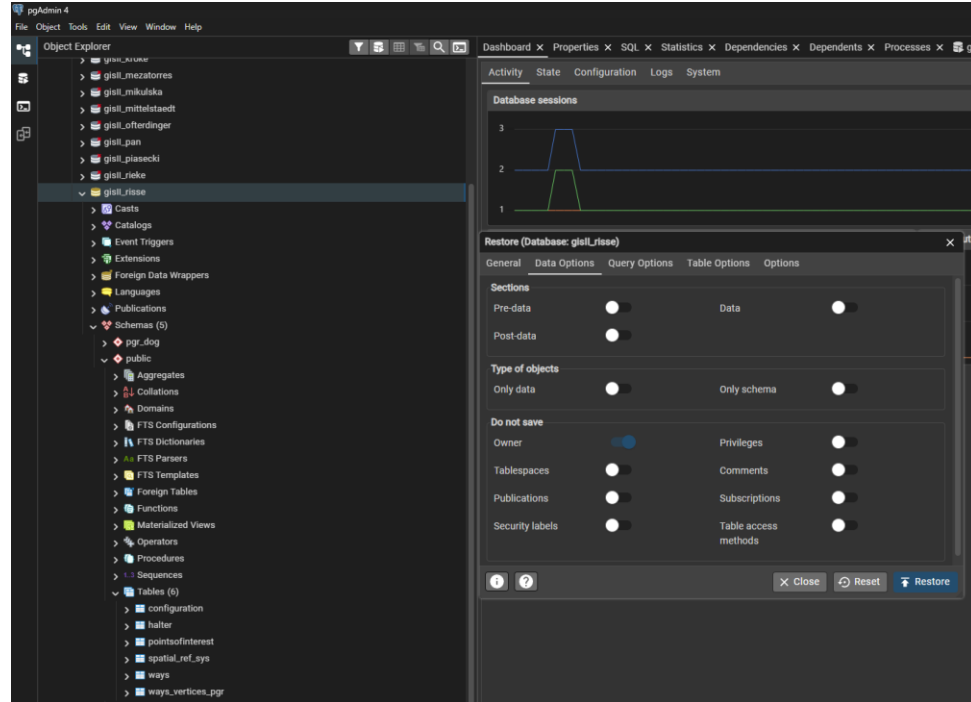
1. Getting started

- pgAdmin öffnen
- Rechtsklick Servers → Register → Server
- Name, Host (edu-map25.geomatik.ruhr-uni-bochum.de), Username: [Nachname], Password: [Matrikelnr.]



2. Datenbank einfügen

- Im Object Explorer: Server → Databases
 - Rechtsklick auf deine DB → Restore
 - Format: Custom or tar, Filename: Pfad zur pgr_dog.dump, unter Data Options: Do not save Owner
 - Status „Failed“ ist ok, sofern unter Schemas → public → Tables diese Tabellen liegen:
 - halter,
 - ways und
 - ways_vertices_pgr
- (evtl. rechtsklick auf Tables → Refresh)



ways_vertices_pgr:
Knoten mit Koordinaten

ways:
Kanten mit zugehörigen Knoten,
Längen und Geschwindkeiten

halter:
Hundebesitzer
mit Koordinaten
und Preisen

$$cost = \frac{ways.length_m}{1000 * ways.maxspeed_forward} * 60 [min]$$

$$node_id = halter.(lon,lat) \leftrightarrow ways_vertices_pgr.(lon,lat)$$

Schemas → public → Tables → Rechtsklick
auf halter → View/Edit Data → First 100 Rows

Wollt ihr jemanden
hinzufügen?



→ Add Row

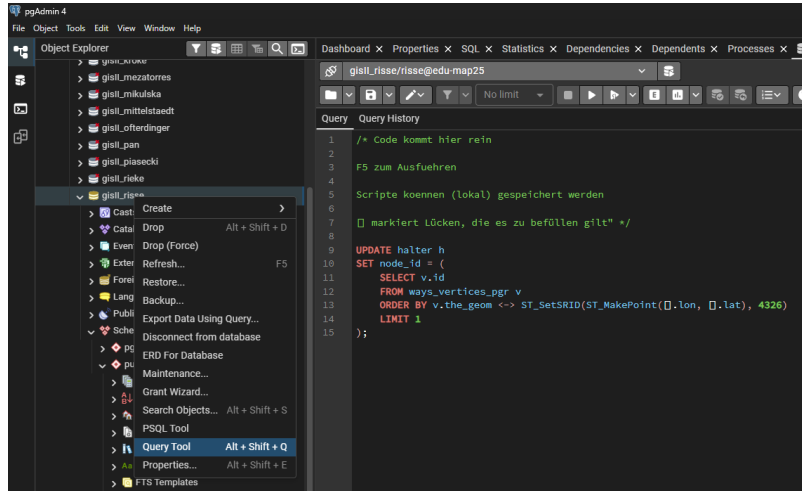


→ Save Changes

Data Output Messages Notifications										
	id [PK] integer	name text	hund text	adresse text	stadtteil text	lon double precision	lat double precision	preis numeric (5,2)	node_id bigint	
1	0	Zuhause	-	Universitaetsstrasse 150, 44803 Bochum	Querenburg	7.2612	51.4453	0.00	[null]	
2	1	Familie Nowak	Rex	Auf dem Backenberg 3, 44801 Bochum	Querenburg	7.268	51.4478	18.00	[null]	
3	2	Herr Yilmaz	Rocky	Hustadttring 40, 44801 Bochum	Querenburg	7.2676	51.4563	8.00	[null]	
4	3	Frau Schmidt	Luna	Brenscheder Str. 12, 44797 Bochum	Wiemelhausen	7.2363	51.4561	9.00	[null]	
5	4	Familie Maier	Max	Castroper Str. 45, 44791 Bochum	Grumme	7.226	51.4851	20.00	[null]	
6	5	Herr Koehler	Rudi	Kortumstrasse 62, 44787 Bochum	Innenstadt	7.2172	51.4798	8.00	[null]	
7	6	Frau Bauer	Mia	Wittener Strasse 200, 44803 Bochum	Altenbochum	7.2443	51.4718	16.00	[null]	
8	7	Familie Heffernan	Arthur	Iserlohner Strasse 15, 44894 Bochum	Langendreer	7.3321	51.4883	22.00	[null]	
9	8	Herr Petrov	Laika	Hammerstrasse 100, 44793 Bochum	Hamme	7.1664	51.4833	12.00	[null]	
10	9	Frau Hoffmann	Chico	Voedestr. 88, 44869 Bochum	Wattenscheid	7.1441	51.4816	28.00	[null]	
11	10	Familie Dogan	Pasta	Hiltroper Landwehr 20, 44805 Bochum	Hiltrop	7.2636	51.5243	24.00	[null]	
12	99	Hundewiese	-	Henriettenstrasse 29, 44793 Bochum	Innenstadt	7.2082	51.4761	0.00	[null]	

4. ways.node_id bestimmen

Unsere Halter haben Koordinaten (lon, lat) – aber pgRouting arbeitet mit Knoten-IDs. Wir suchen aus ways_vertices_pgr für jeden Halter den nächstgelegenen Knoten und speichern dessen ID in halter.node_id.



Data Output		Messages		Notifications					
									
	id [PK] integer	name text	hund text	adresse text	stadtteil text	lon double precision	lat double precision	preis numeric (10,2)	node_id bigint
1	0	Zuhause	-	Universitaetss...	Querenburg	7.2612	51.4453	0.00	69190
2	1	Familie Nowak	Rex	Auf dem Back...	Querenburg	7.268	51.4478	18.00	61718
3	2	Herr Yilmaz	Rocky	Hustadtring 4...	Querenburg	7.2676	51.4563	8.00	137072
4	3	Frau Schmidt	Luna	Brenscheder S...	Wiemelhau...	7.2363	51.4561	9.00	68873
5	4	Familie Maier	Max	Castroper Str. ...	Grumme	7.226	51.4851	20.00	71986
6	5	Herr Koehler	Rudi	Kortumstrasse...	Innenstadt	7.2172	51.4798	8.00	194409
7	6	Frau Bauer	Mia	Wittener Stras...	Altenbochum	7.2443	51.4718	16.00	69973
8	7	Familie Heffern...	Arth...	Iserlohner Stra...	Langendreer	7.3321	51.4883	22.00	171644
9	8	Herr Petrov	Laika	Hammerstras...	Hamme	7.1664	51.4833	12.00	32237
10	9	Frau Hoffmann	Chico	Voedestr. 88, 4...	Wattensche...	7.1441	51.4816	28.00	108348
11	10	Familie Dogan	Pasta	Hiltroper Land...	Hiltrop	7.2636	51.5243	24.00	50997
12	99	Hundewiese	-	Henriettenstra...	Innenstadt	7.2082	51.4761	0.00	105187

```
UPDATE halter h
SET node_id = (
    SELECT v.id
    FROM ways_vertices_pgr v
    ORDER BY v.the_geom <-> ST_SetSRID(
        ST_MakePoint(h.lon, h.lat), 4326)
    LIMIT 1
);
```

5. Kostenmatrix berechnen

DijkstraCostMatrix berechnet für jedes Knotenpaar aus Tabelle halter die kürzeste Fahrzeit – das Ergebnis ist eine vollständige (Zeit)Kostenmatrix.

Vervollständigt den Code einschließlich des Arrays mit den halter.ids von Start, Ziel und euren auserwählten Kunden.

```
SELECT * FROM pgr_dijkstraCostMatrix(  
    'SELECT gid AS id, source, target,  
        ways.length_m / 1000.0 / □ * 60.0 AS cost,  
        CASE WHEN reverse_cost > 0  
            THEN ways.length_m / 1000.0 / □ * 60.0  
            ELSE -1  
        END AS reverse_cost  
    FROM ways',  
    ARRAY(SELECT node_id FROM halter  
        WHERE id = ANY(ARRAY[□, □, □, □, □, □])),  
    directed := true  
);
```

Data Output				Messages	Notifications
	start_vid bigint	end_vid bigint	agg_cost double precision		
1	68873	69190	3.2017344644476418		
2	68873	105187	4.638117465071599		
3	68873	108348	8.143315924052459		
4	68873	171644	9.806675973671718		
5	68873	194409	4.49442688181564		
6	69190	68873	3.274578358164925		
7	69190	105187	7.1659548082439954		
8	69190	108348	10.671153267224854		
9	69190	171644	8.937912048891201		
10	69190	194409	7.0222642249880405		
11	105187	68873	4.544692154500091		
12	105187	69190	7.122024817202436		
13	105187	108348	6.276859135946241		
14	105187	171644	10.05858235438249		
15	105187	194409	1.707770863462437		
16	108348	68873	8.251623677628407		
17	108348	69190	10.82895634033075		
18	108348	105187	6.3296445629836455		
19	108348	171644	15.365193738793073		
20	108348	194409	7.347898359054631		
21	171644	68873	9.520379939946634		
22	171644	69190	9.808713231040123		

6. TSP für ideale Route

pgr_TSP findet die optimale Reihenfolge der Knoten – also in welcher Reihenfolge sollen wir die Halter anfahren, um die Gesamtfahrzeit zu minimieren.

Beachtet dabei: TSP berechnet Runden, daher ist die letzte Station immer der Ausgangspunkt (hier seq 7), den ignorieren wir.

```
SELECT seq, node, cost, agg_cost
FROM pgr_TSP(
    $$SELECT * FROM pgr_dijkstraCostMatrix(
        'SELECT gid AS id, source, target,
            ways.length_m / 1000.0 / 60 * 60.0 AS cost,
            CASE WHEN ways.reverse_cost > 0
                THEN ways.length_m / 1000.0 / 60 * 60.0
                ELSE -1
            END AS reverse_cost
        FROM ways',
        ARRAY(SELECT node_id FROM halter
            WHERE id = ANY(ARRAY[□, □, □, □, □, □])),
        directed := true
    )$$,
    start_id := (SELECT node_id FROM halter WHERE id = □),
    end_id := (SELECT node_id FROM halter WHERE id = □)
);
```

Data Output		Messages		Notifications	
	seq integer	node bigint	cost double precision	agg_cost double precision	
1	1	69190	0	0	
2	2	137072	1.9064545835129048	1.9064545835129048	
3	3	171644	8.6600377681583	10.566492351671204	
4	4	108348	13.324488964206251	23.890981315877454	
5	5	194409	7.347898359054631	31.238879674932086	
6	6	105187	1.707770863462437	32.94665053839452	
7	7	69190	7.122024817202436	40.06867535559696	

7. tsp() als Funktion

Da wir TSP noch öfter berechnen müssen, speichern wir das in einer Funktion. Funktionen werden einmalig erstellt und sind dann dauerhaft in der DB unter Schemas → public → Functions gespeichert.

```
CREATE OR REPLACE FUNCTION tsp(halter_ids int[], start_id int, end_id int)
RETURNS TABLE(seq int, node bigint, cost float, agg_cost float) AS $func$
DECLARE
    knoten bigint[];
    matrix text;
BEGIN
    SELECT ARRAY(SELECT node_id FROM halter
                WHERE id = ANY(halter_ids)) INTO knoten;

    matrix := 'SELECT * FROM pgr_dijkstraCostMatrix(
                ''SELECT gid AS id, source, target,
                    ways.length_m / 1000.0 / ways.maxspeed_forward * 60.0 AS cost,
                    CASE WHEN ways.reverse_cost > 0
                        THEN ways.length_m / 1000.0 /
                            ways.maxspeed_forward * 60.0
                        ELSE -1
                    END AS reverse_cost
                FROM ways'',
                ARRAY[' || array_to_string(knoten, ',') || '],
                directed := true)';

    RETURN QUERY
    SELECT t.seq, t.node, t.cost, t.agg_cost
    FROM pgr_TSP(
        matrix,
        start_id := (SELECT node_id FROM halter WHERE id = start_id),
        end_id    := (SELECT node_id FROM halter WHERE id = end_id)
    ) t;
END;
$func$ LANGUAGE plpgsql;
```


text

```
WITH tsp_hinweg AS (  
    SELECT node, seq FROM tsp(ARRAY[0,1,3,7,10,99], 0, 99)  
    ORDER BY seq  
)  
SELECT ST_Collect(ways.the_geom) AS route  
FROM pgr_dijkstraVia(  
    'SELECT gid AS id, source, target,  
        ways.length_m / 1000.0 / ways.maxspeed_forward * 60.0 AS cost,  
        CASE WHEN ways.reverse_cost > 0  
            THEN ways.length_m / 1000.0 /  
                ways.maxspeed_forward * 60.0  
            ELSE -1  
        END AS reverse_cost  
    FROM ways',  
    ARRAY(SELECT node FROM tsp_hinweg  
        WHERE seq < (SELECT MAX(seq) FROM tsp_hinweg)),  
    directed := true  
) AS route  
JOIN ways ON ways.gid = route.edge  
WHERE route.edge != -1;
```

7. TSP für Hin- & Rückweg

Um die Hunde auch wieder abzuliefern, führen wir pgr_TSP zweimal aus. Mit UNION ALL werden beide Ergebnisse in einer Tabelle zusammengefasst.

```
SELECT 'Hinweg' AS richtung, seq, node, cost, agg_cost
FROM pgr_TSP(
    $$SELECT * FROM pgr_dijkstraCostMatrix(
        'SELECT gid AS id, source, target,
            ways.length_m / 1000.0 / □ * 60.0 AS cost,
            CASE WHEN ways.reverse_cost > 0
                THEN ways.length_m / 1000.0 / □ * 60.0
                ELSE -1
            END AS reverse_cost
        FROM ways',
        ARRAY(SELECT node_id FROM halter
            WHERE id = ANY(ARRAY[□, □, □, □, □, □])),
        directed := true
    )$$,
    start_id := (SELECT node_id FROM halter WHERE id = □),
    end_id := (SELECT node_id FROM halter WHERE id = □)
)
UNION ALL
SELECT 'Rückweg' AS richtung, seq, node, cost, agg_cost
FROM pgr_TSP(
    $$...$$, -- start_id und end_id tauschen!
    start_id := (SELECT node_id FROM halter WHERE id = □),
    end_id := (SELECT node_id FROM halter WHERE id = □)
)
ORDER BY richtung, seq;
```

	richtung text	seq integer	node bigint	cost double precision	agg_cost double precision
1	Hinweg	1	69190	0	0
2	Hinweg	2	61718	0.972012847441704	0.972012847441704
3	Hinweg	3	68873	3.979835230204347	4.951848077646051
4	Hinweg	4	50997	8.516348126802805	13.468196204448857
5	Hinweg	5	171644	9.22288208256941	22.691078287018264
6	Hinweg	6	105187	10.05858235438249	32.74966064140075
7	Hinweg	7	69190	7.122024817202436	39.87168545860319
8	Rückweg	1	105187	0	0
9	Rückweg	2	68873	4.544692154500091	4.544692154500091
10	Rückweg	3	50997	8.516348126802805	13.061040281302898
11	Rückweg	4	171644	9.22288208256941	22.283922363872307
12	Rückweg	5	61718	8.550903593423786	30.834825957296093
13	Rückweg	6	69190	0.972012847441704	31.806838804737797
14	Rückweg	7	105187	7.122024817202436	38.92886362194023

8. Route visualisieren

pgr_dijkstraVia berechnet die genaue Strecke entlang des Straßennetzes durch alle Knoten in der TSP-Reihenfolge. ST_Collect fasst alle Straßenabschnitte zu einer Linie zusammen – sichtbar im Geometry Viewer

```
SELECT 'Hinweg' AS richtung, seq, node, cost, agg_cost
FROM pgr_TSP(
    $$SELECT * FROM pgr_dijkstraCostMatrix(
        'SELECT gid AS id, source, target,
            ways.length_m / 1000.0 / 60 * 60.0 AS cost,
            CASE WHEN ways.reverse_cost > 0
                THEN ways.length_m / 1000.0 / 60 * 60.0
            ELSE -1
            END AS reverse_cost
        FROM ways',
        ARRAY(SELECT node_id FROM halter
            WHERE id = ANY(ARRAY[1, 2, 3, 4, 5, 6])),
        directed := true
    )$$,
    start_id := (SELECT node_id FROM halter WHERE id = 1),
    end_id := (SELECT node_id FROM halter WHERE id = 6)
)
UNION ALL
SELECT 'Rückweg' AS richtung, seq, node, cost, agg_cost
FROM pgr_TSP(
    $$...$$, -- start_id und end_id tauschen!
    start_id := (SELECT node_id FROM halter WHERE id = 6),
    end_id := (SELECT node_id FROM halter WHERE id = 1)
)
ORDER BY richtung, seq;
```

8. Route visualisieren

pgr_dijkstraVia berechnet die genaue Strecke entlang des Straßennetzes durch alle Knoten in der TSP-Reihenfolge. ST_Collect fasst alle Straßenabschnitte zu einer Linie zusammen – sichtbar im Geometry Viewer.

```
UPDATE halter h
SET node_id = (
SELECT v.id
FROM ways_vertices_pgr v
ORDER BY v.the_geom <-> ST_SetSRID(
ST_MakePoint(□.lon, □.lat), 4326)
LIMIT 1
);
```